

Câu lệnh điều kiện

Nguyễn Huyền Châu – AI Lab/TLU

Nội dung

- Tạo ra các giá trị bool
- Các câu lệnh if
- Câu lệnh switch
- Các ứng dụng của câu lệnh điều kiện
- Chủ đề mở rộng

Tạo ra các giá trị bool

- ❑ Toán tử quan hệ
- ❑ Toán tử logic
- ❑ Thứ tự ưu tiên tính toán
- ❑ Kỹ thuật kiểm tra khoảng số
- ❑ Các lưu ý với kiểu bool

Toán tử quan hệ

- Xác định quan hệ giữa 2 toán hạng thuộc **kiểu so sánh được**:

> Lớn hơn

< Nhỏ hơn

>= Lớn hơn hoặc bằng

<= Nhỏ hơn hoặc bằng

== **Bằng (khá dễ nhầm với toán tử gán)**

!= Không bằng

- Sẽ tạo ra biểu thức quan hệ có kiểu bool:
 - ❑ $12 > 5$ là một biểu thức quan hệ, có giá trị **true**
 - ❑ $7 \leq 5$ là một biểu thức quan hệ, có giá trị **false**
 - ❑ Với x bằng 10 ta có $x == 10$ là **true** và $x == 8$ là **false**
 - ❑ Giá trị bool sẽ gán được cho mọi biến bool, vd: **bool** test = $x < y$;

Toán tử logic

- Dùng trên toán hạng bool tạo ra biểu thức logic cũng kiểu bool

& &	AND	true nếu toán hạng trái và phải đều là true
	OR	true nếu ít nhất một trong hai toán hạng true
!	NOT	trả về true khi toán hạng là false, và ngược lại

- => Đến giờ ta đã biết những gì có thể có kiểu bool:
 - Biến, hằng khai báo kiểu bool, literal kiểu bool (**true/false**)
 - Biểu thức quan hệ cũng cho kết quả kiểu bool : $a > b$
 - Sự kết hợp những thứ trên qua toán tử logic: $(a > b) \&\& (a > c)$
- Giá trị kiểu bool rất quan trọng trong C++, nó dùng làm điều kiện trong lệnh if và vòng lặp

Ví dụ: Toán tử quan hệ, toán tử logic

```
int x = 12, y = 5, z = -4;
```

<code>(x > y) && (y > z)</code>	
<code>(x > y) && (z > y)</code>	
<code>(x <= z) (y == z)</code>	
<code>(x <= z) (y != z)</code>	
<code>! (x >= z)</code>	

Thứ tự ưu tiên khi tính toán

1. Các phép số học:

Phép – một ngôi thì mạnh hơn phép *,/, % rồi mạnh hơn +, -

2. Phép phủ định !

3. Các phép quan hệ:

Các phép <, <=, >, >= mạnh hơn các phép ==, !=

4. Các phép logic còn lại:

Phép && mạnh hơn phép ||

■ Thực hành: các biểu thức dưới có kiểu gì và tính theo thứ tự nào?

❑ $-x * y + 3$

❑ $!x + 3*y = ! (x + (3 * y))$

❑ $!x > y + 5$

❑ $x > 3 || ! (x \% 3 == 0) \&\& x < 10$

Toán tử quan hệ và toán tử logic giúp kiểm tra các khoảng số

- Có thể kiểm tra một giá trị có thuộc một khoảng:

```
if (grade >= 0 && grade <= 100)
```

```
    cout << "Valid grade";
```

- Cũng có thể kiểm tra một giá trị có nằm ngoài một khoảng:

```
if (grade < 0 || grade > 100)
```

```
    cout << "Invalid grade";
```

- Cần tách thành hai biểu thức quan hệ nối bởi toán tử logic:

```
if (0 <= grade <= 100) // still compiles, doesn't work
```


Các lưu ý về kiểu bool

- Khi cần, C++ có thể ép kiểu bool thành kiểu số và ngược lại:
 - `false` trở thành 0 và `true` thành 1.
 - 0 thành `false` và khác 0 thành `true`.
- => Mọi biểu thức số học đều chuyển được thành kiểu bool
 - `if (x != 0)` và `if (x==0)` có thể viết tắt thành `??`
 - VD: `if (a %3)` tương đương `if ... ??`
- => Viết nhầm biểu thức logic thành biểu thức số học thì C++ không báo lỗi, thường gặp là viết nhầm `==` thành `=`:
 - Biểu thức `x == 3` nhận `true/false` tùy vào x
 - Biểu thức `x = 3` sẽ có giá trị 3, luôn là `true` (còn `x=0` thì `??`)

Ví dụ: Nhầm phép `==` và phép `=`

```
#include <iostream>
using namespace std;
int main(){
    int payCode;
    cout << "\nPay code: ";
    cin >> payCode;

    cout << "\nTest payCode == 4:" << endl;
    if (payCode == 4)
        cout << "You get a bonus!\n";

    cout << "\nTest 4 == payCode:" << endl;
    if (4 == payCode)
        cout << "You get a bonus!\n";

    cout << "\nTest payCode = 4:" << endl;
    if (payCode = 4)
        cout << "You get a bonus!\n";

    return 0;
}
```

Pay code: 5

Test payCode == 4:

Test 4 == payCode:

Test payCode = 4:
You get a bonus!

Pay code: 4

Test payCode == 4:
You get a bonus!

Test 4 == payCode:
You get a bonus!

Test payCode = 4:
You get a bonus!

Nhầm phép `==` và phép `=`

```
if (payCode == 4) cout << " You get a bonus!"
```

- nếu mã tiền lương (paycode) là 4 thì được thưởng

```
if (payCode = 4) cout << " You get a bonus!"
```

- paycode sẽ có giá trị 4 (không cần biết giá trị trước đó)

- giá trị biểu thức gán chính là 4, nên true (vì 4 khác 0)

- trường hợp nào cũng được thưởng

- Mẹo: nên viết `(4==payCode)` sẽ tránh nhầm ??

- lvalue: thứ được quyền ở bên trái phép gán (vd: biến): `x = 4`

- rvalue: chỉ xuất hiện bên phải phép gán (hằng, literal)

- Không thể viết `4 = x`, nếu viết C++ sẽ báo lỗi, nhờ vậy ta quay lại sửa

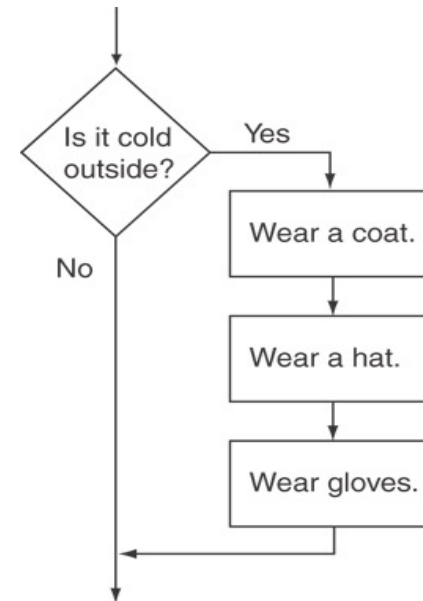
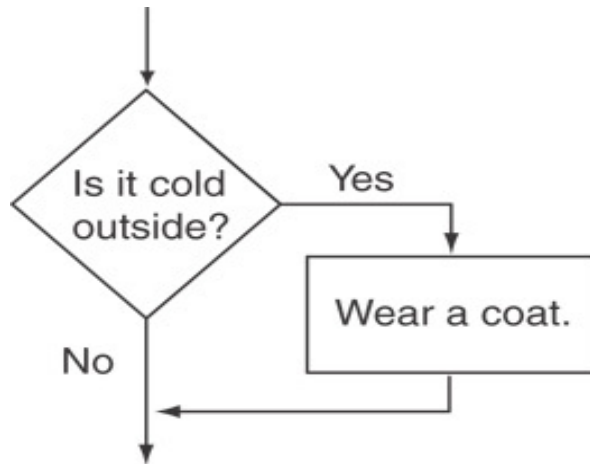
- lvalue có thể dùng như rvalue, chiều ngược lại không thể

Các kiểu lệnh if

- ❑ Lệnh if
- ❑ Lệnh if/else
- ❑ Lệnh đơn vs. lệnh phức
- ❑ Lệnh lồng
- ❑ Một dạng lồng hữu ích: if/else if

Các câu lệnh lựa chọn

- Sẽ thực hiện hành động nếu một điều kiện nào đó đúng
- Giống ngoài đời, ta ra quyết định tùy theo hoàn cảnh
 - “Nếu trời lạnh, mặc áo khoác.”
 - “Nếu trời lạnh, mặc áo, rồi đội mũ, rồi đeo găng”
- Có nhiều loại : if, if/else, biểu thức điều kiện, if/else if, switch



Lệnh if

- Cho phép chọn giữa các tuyến hành động khác nhau

if (điều kiện)

<câu lệnh>

**Điều kiện luôn bao
trong dấu ngoặc ()**

if student's grade is greater than or equal to 60

print "Passed"



```
if (score >= 60)  
    cout << "Passed";
```

- Nếu điều kiện thoả mãn (có giá trị true)
 - Lệnh print được thực hiện, chương trình chạy tiếp lệnh tiếp theo
- Nếu điều kiện không thoả mãn (có giá trị false)
 - Lệnh print bị bỏ qua, chương trình chạy tiếp.
- <Câu lệnh> hiểu là lệnh đơn hoặc khối lệnh

Thực hành: Lệnh if

```
// Nhập vào cân nặng, tính trọng lượng = cân nặng * 9.8
// Nếu trọng lượng < 10, thông báo vật quá nhẹ
// Nếu trọng lượng > 100, thông báo vật quá nặng
#include<iostream>
using namespace std;
int main(){
    const float G(9.8);
    float khoiluong, trongluong;
    cout << "Khoi luong: ";
    cin >> khoiluong;
    trongluong = khoiluong * G;
    if (trongluong<10) cout << "Vat qua nhẹ \n";
    if (trongluong>100) cout << "Vat qua nặng \n";
    return 0;
}
```

Khoi luong: 1
Vat qua nhẹ

Khoi luong: 12
Vat qua nặng

Khoi luong: 7

Giá trị không bị bắt bởi if nào,
nên nhảy qua cả 2 lệnh in

Lệnh if/else

- **if**: thực hiện hành động nếu điều kiện thoả mãn
- **if/else**: thực hiện hành động khác nhau tùy theo điều kiện thoả mãn hay không
- Mã giả
- Mã C++:

```
if student's grade is greater than or equal to 60  
    print "Passed"  
else  
    print "Failed"
```

```
if (grade >= 60)  
|   cout << "Passed";  
else  
|   cout << "Failed";
```


Thực hành: Lệnh if/else

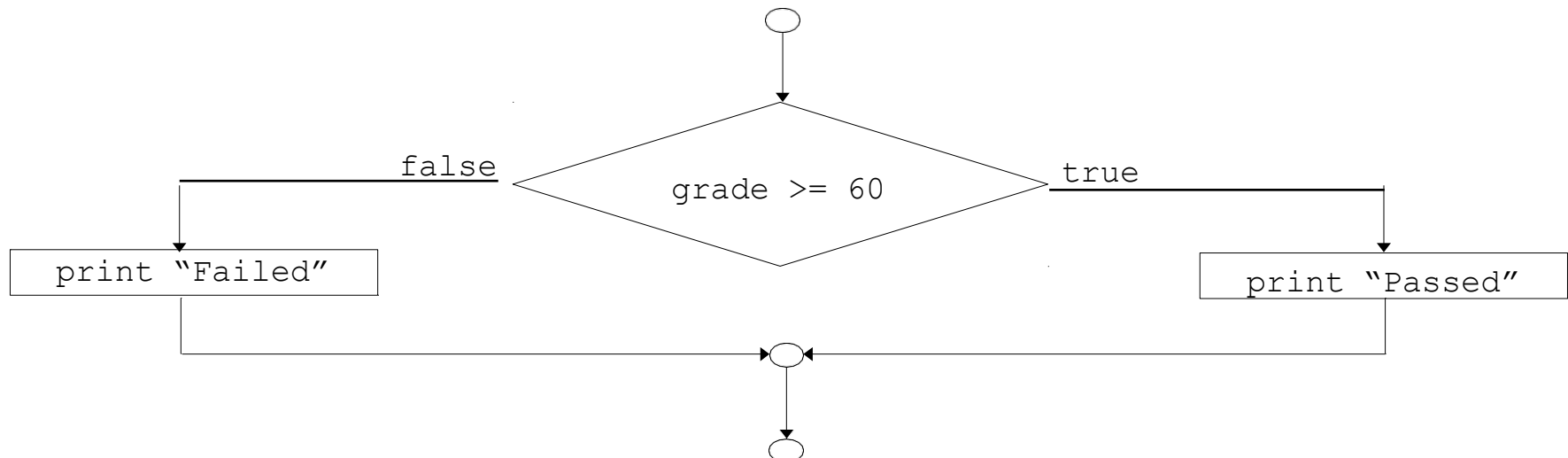
```
// Ngày phép thuật có đặc điểm: tháng * ngày = năm (tính 2 chữ số cuối)
// Nhập vào ngày tháng năm (lấy 2 chữ số cuối), in ra có phải ngày phép thuật không
#include <iostream>
using namespace std;
int main(){
    int ngay, thang, nam;
    cout << "Nhap ngay, thang, nam:\n";
    cin >> ngay >> thang >> nam;
    if (ngay * thang == nam) cout << "Ngày phép thuật";
    else cout << "Không phải ngày phép thuật";
    cout << endl;
    return 0;
}
```

Lệnh if/else vs. toán tử điều kiện

- Biểu thức điều kiện gồm 3 tham số xen giữa **?** và **:**
(điều kiện **?** giá trị nếu **true** : giá trị nếu **false**)
- Thay thế các if/else ngắn, vd if/else slide trước viết lại được như sau:

```
cout << (grade >= 60 ? "Passed" : "Failed");
```

Condition Value if true Value if false



Thực hành: Toán tử điều kiện

// Viết chương trình yêu cầu người dùng nhập vào hai số, sử dụng toán tử điều kiện (toán tử ? :)
// để in ra số lớn hơn và số nhỏ hơn trong 2 số.

```
#include <iostream>  
using namespace std;
```

```
int main(){  
    float a, b;  
    cout << "Nhập 2 số: "; cin >> a >> b;  
    cout << "Số lớn hơn là " << (a>=b? "a" : "b") << endl;  
    (a>=b)? cout << "Số lớn hơn là a" << endl : cout << "Số lớn hơn là b" << endl;  
    (a>=b)? cout << "Số lớn hơn là a", cout << endl : (cout << "Số lớn hơn là b", cout << endl);  
    return 0;  
}
```

3 câu lệnh này hoàn toàn tương đương

```
Nhập 2 số: 5 10  
Số lớn hơn là b  
Số lớn hơn là b  
Số lớn hơn là b
```

Biểu thức điều kiện ngoài có thể khai báo giá trị, còn có thể gọi lệnh, thậm chí nhiều lệnh, miễn các lệnh cùng nhóm thì tách nhau bởi dấu phẩy và các lệnh ứng với điều kiện false cần đặt trong ngoặc ()

Lệnh if, if/else kiểu đơn và phức

- Lệnh đơn: nếu điều kiện đúng/sai chỉ thực hiện 1 lệnh
- Lệnh phức: nếu điều kiện đúng/sai thì thực hiện 1 tập lệnh
 - tập lệnh này cần bao bên trong cặp ngoặc {}

```
if (score >= 90){  
    cout << "Passed\n";  
    cout << "You also got grade A!";  
}
```

vs.

```
if (score >= 90)  
    cout << "Passed. \n";  
    cout << "You also got grade A! \n";
```

Score $\geq$ 90 thì grade A

Luôn grade A với mọi score

```
if (score >= 50)  
    cout << "Passed. \n";  
else {  
    cout << "Failed. \n";  
    cout << "You must take this course again! \n";  
}
```

vs.

```
if (score >= 50)  
    cout << "Passed. \n";  
else  
    cout << "Failed. \n";  
    cout << "You must take this course again! \n";
```

Score $<$ 50 thì học lại

Luôn học lại với mọi score

Lệnh if lồng

- Là lệnh if, if/else đặt trong thân lệnh if khác
- Khi muốn kiểm tra tiếp các điều kiện khác:
 - Ngoài là người Việt Nam, còn cần là dân tộc Kinh, giới tính nam, ...
- if cha là lệnh phức thì if con nằm trong cặp {}

```
if (x%2==0){  
    if (x%3==0) cout << "x chia het cho 6";  
    cout << "x chia het cho 2";  
}
```

- if cha là lệnh đơn thì không cần {}, còn viết được hai if trên 1 dòng

```
if (x%2==0)  
    if (x%3==0) cout << "x chia het cho 6";  
  
if (x%2==0) if (x%3==0) cout << "x chia het cho 6";
```

Thực hành: Lệnh if lồng

// Nhập số calo và số g chất béo của thức ăn, tính tỷ lệ calo từ chất béo nếu tỷ lệ
// nhỏ hơn 30% thì thông báo là thức ăn ít béo báo lỗi nếu nhập số âm hay nếu số calo
// từ chất béo lớn hơn tổng số calo biết 1 gam chất béo cung cấp 9 calo

```
#include <iostream>
using namespace std;
int main()
{
    const float ONE_GRAM = 9;
    float calories, fat, fatPercent;
    cout << "So calories va so gam chat beo: ";
    cin >> calories >> fat;

    if (fat < 0 || calories < 0 || fat * ONE_GRAM > calories) cout << "Nhap loi!\n";
    else {
        fatPercent = ONE_GRAM * fat / calories ;
        cout << "Ty le chat beo: " << fatPercent * 100 << "%\n";
        if (fatPercent <= 0.3) cout << "Day la thuc pham it beo!" << endl;
    }
    return 0;
}
```

Chỉ cần 1 trong 3
điều kiện này xảy ra
thì sẽ không xử lý
dữ liệu đầu vào

Lệnh if/else if

- Là if lồng nhưng là lệnh đơn, dùng để kiểm tra các điều kiện rời nhau
- một điều kiện đúng thì mọi điều kiện khác sai nên bị bỏ qua

if student's grade is greater than or equal to 90

Print "A"

else

if student's grade is greater than or equal to 80

Print "B"

else

if student's grade is greater than or equal to 70

Print "C"

else

if student's grade is greater than or equal to 60

Print "D"

else

Print "F"

```
| if (grade >= 90)
|     cout << "A";
| else if (grade >= 80)
|     cout << "B";
| else if (grade >= 70)
|     cout << "C";
| else if (grade >= 60)
|     cout << "D";
| else cout << "F";
```

Lệnh if/else if

- phù hợp bài toán “chia miền”: thực hiện hành động tùy theo dữ liệu thuộc miền nào trong nhiều miền không giao nhau
- else cuối là tùy chọn, có thể thêm vào để bắt lỗi



```
if (grade >= 90)
    cout << "A";
else if (grade >= 80)
    cout << "B";
else if (grade >= 70)
    cout << "C";
else if (grade >= 60)
    cout << "D";
else    cout << "F";
```


Ví dụ: Dùng else cuối để bắt lỗi

```
21 // Determine the letter grade.
22 if (testScore >= A_SCORE)
23     cout << "Your grade is A.\n";
24 else if (testScore >= B_SCORE)
25     cout << "Your grade is B.\n";
26 else if (testScore >= C_SCORE)
27     cout << "Your grade is C.\n";
28 else if (testScore >= D_SCORE)
29     cout << "Your grade is D.\n";
30 else if (testScore >= 0)
31     cout << "Your grade is F.\n";
32 else
33     cout << "Invalid test score.\n";
```

else cuối để
bắt giá trị
không hợp lệ



Thực hành: Lệnh if/else if

// Yêu cầu người dùng nhập chiều dài và rộng của 2 hình chữ nhật, rồi cho biết
// hình nào có diện tích lớn hơn, nếu bằng nhau thì thông báo bằng nhau

```
#include <iostream>
using namespace std;
int main(){
    float dai1, rong1, dai2, rong2;
    cout << "Nhap dai, rong cua hinh 1: ";
    cin >> dai1 >> rong1;
    cout << "Nhap dai, rong cua hinh 2: ";
    cin >> dai2 >> rong2;

    float dientich1 = dai1 * rong1, dientich2 = dai2 * rong2;

    if (dientich1 > dientich2) cout << "Hinh 1 lon hon hinh 2\n";
    else if (dientich1 < dientich2) cout << "Hinh 2 lon hon hinh 1\n";
    else cout << "Hai hinh bang nhau\n";
    return 0;
}
```

else cuối bắt mọi trường hợp còn lại, chính là trường hợp bằng, nên không cần thêm if

Thực hành: Lệnh if/else if

// Chỉ số BMI = cân nặng (kg)/ bình phương của chiều cao. Nếu BMI lớn hơn 25 là thừa cân,
// nhỏ hơn 18.5 là gầy, từ 18.5 đến 25 là bình thường, nhỏ hơn 16 là rất gầy. Nhập chiều cao
// và cân nặng một người rồi cho biết họ đang ở mức cân nặng nào trong 4 loại trên

```
#include <iostream>
using namespace std;
int main(){
    float chieucao, kannang, bmi;
    cout << "Nhap chieu cao, can nang:\n";
    cin >> chieucao >> kannang;

    bmi = chieucao / (kannang * kannang);

    if (bmi > 25) cout << "Thua can";
    else if (bmi > 18.5) cout << "Binh thuong";
    else if (bmi > 16) cout << "Gay";
    else cout << "Rat gay";
    cout << endl;
    return 0;
}
```

Các lưu ý với lệnh if

- Đừng thêm ; sau biểu thức điều kiện:

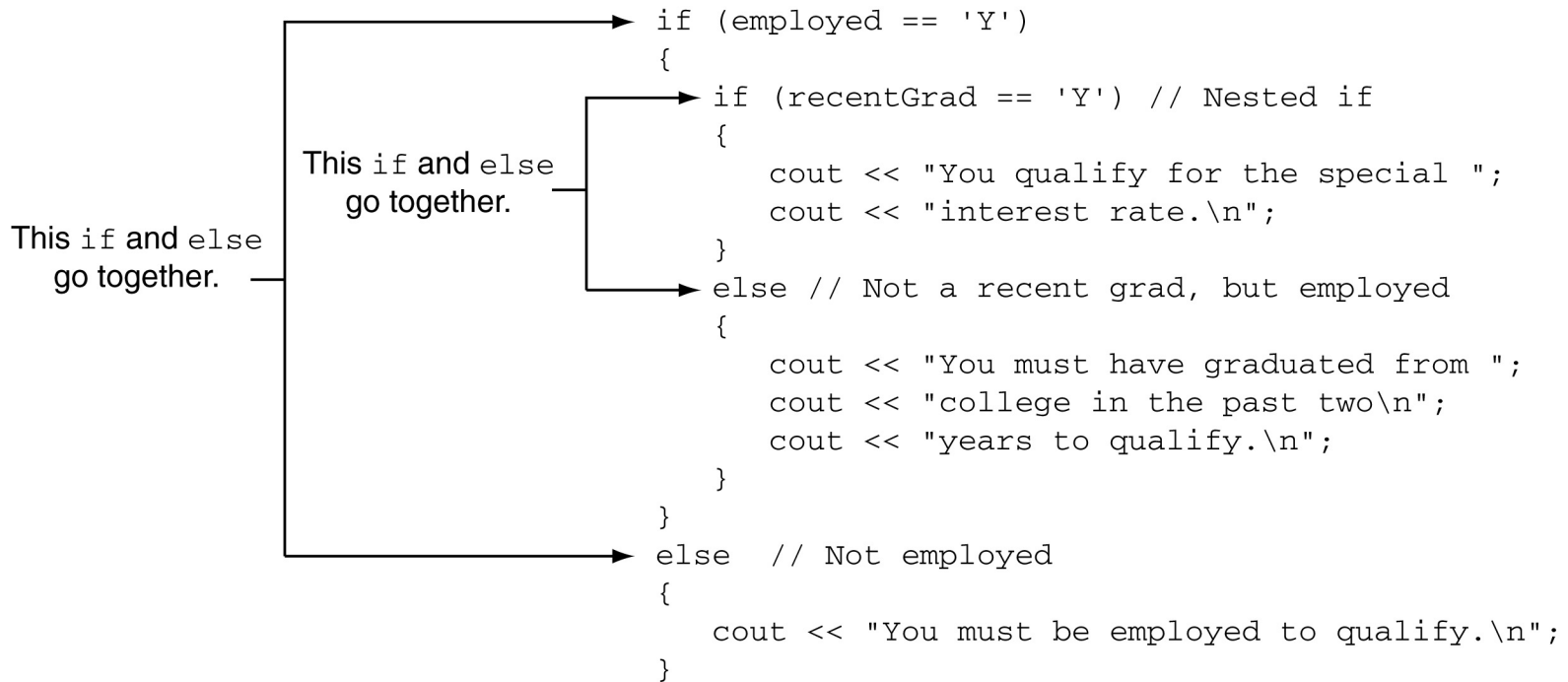
```
if (score >= 50);  
|   cout << "Passed. \n";
```

Score: 30
Passed.

*Luôn in ra "Passed"
với mọi score*

- Bởi vì ; đã đủ là 1 câu lệnh
- Chú ý lệnh đơn hay phức
- Biểu thức điều kiện luôn trong ngoặc ()
- 0 là false; mọi giá trị khác 0 hiểu là true
- Trừ khi dùng if/else if, còn if và else luôn nên đóng thẳng

Lùi đầu dòng phù hợp



Lệnh switch

- ❑ Cú pháp lệnh switch
- ❑ Cách chạy của lệnh switch
- ❑ Các lưu ý với lệnh switch

Câu lệnh đa lựa chọn switch

- if/else if: chọn 1 miền trong nhiều miền không giao nhau
- switch: sẽ chọn 1 giá trị trong nhiều giá trị
 - Mỗi giá trị ứng với một nhãn case:
 - nhãn default không bắt buộc, dù giúp bắt lỗi (giống else cuối)

*choice có thể là biến lẫn hằng hay biểu thức,
còn value phải là giá trị hằng (literal, hằng, biểu thức hằng)*

```
switch (choice) {  
    case value1:           // dùng khi choice == value1  
        statements        // cần thiết để thoát khỏi switch  
        break;  
  
    case value2:           // dùng khi choice == value2 hoặc == value3  
    case value3:  
        statements  
        break;  
  
    default:               // dùng khi choice không khớp nhãn nào  
        statements  
        break;  
}
```

Ví dụ: Câu lệnh switch

/* Yêu cầu người dùng nhập vào một số tự nhiên trong khoảng từ 1 đến 10, sử dụng câu lệnh switch để in ra dạng số La Mã tương ứng của số đó. */

```
#include <iostream>
using namespace std;
int main(){
    int so;
    cout << "Nhập số (1-10): ";
    cin >> so;
    switch (so){
        case 1: cout << "I"; break;
        case 2: cout << "II"; break;
        case 3: cout << "III"; break;
        case 4: cout << "IV"; break;
        case 5: cout << "V"; break;
        case 6: cout << "VI"; break;
        case 7: cout << "VII"; break;
        case 8: cout << "VIII"; break;
        case 9: cout << "IX"; break;
        case 10: cout << "X"; break;
        default: cout << "Không hợp lệ";
    }
    cout << endl;
    return 0;
}
```

Nhập số (1-10): 6
VI

Nhập số (1-10): 12
Không hợp lệ

Ví dụ: Tại sao cần break

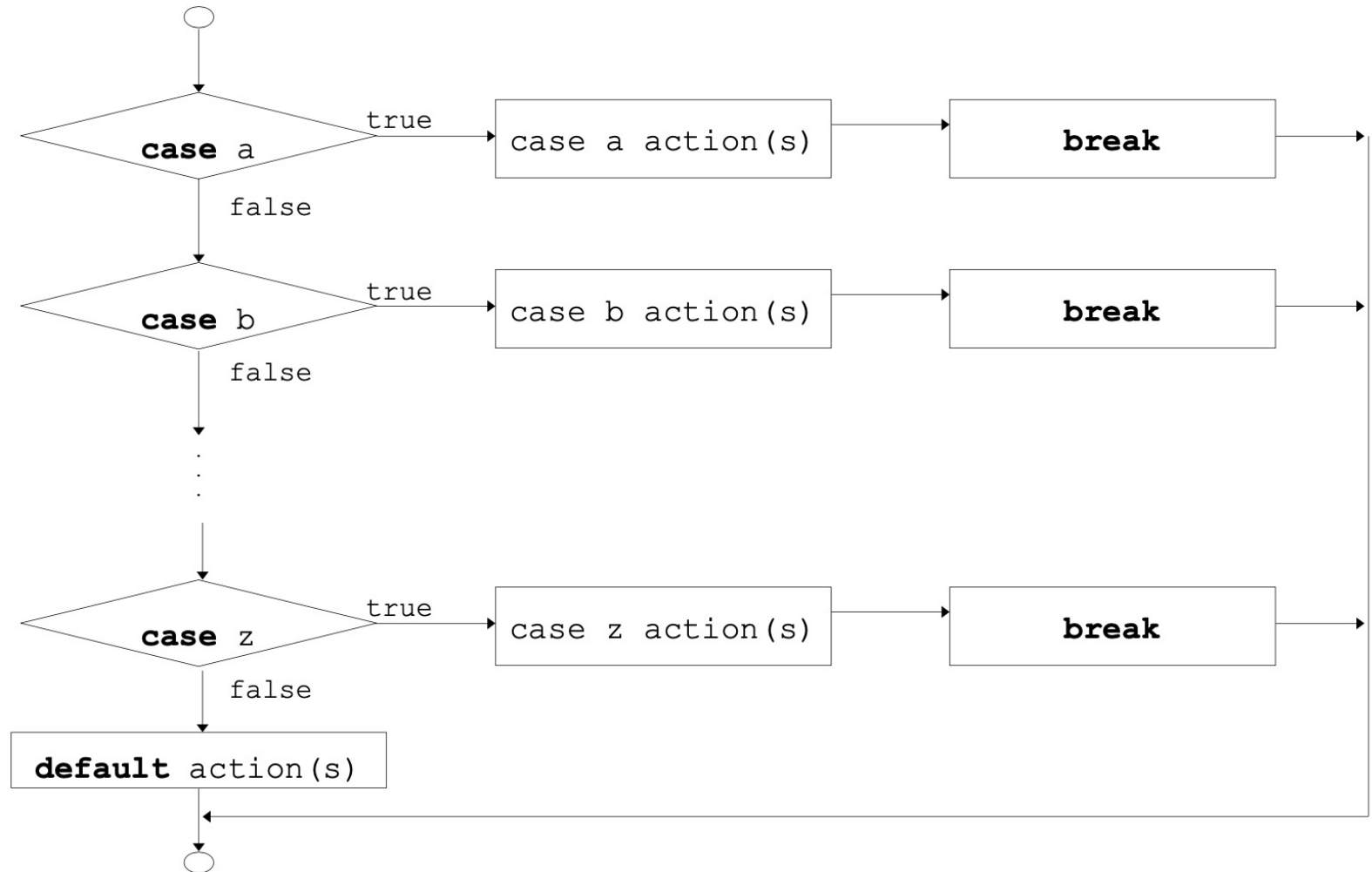
/* Yêu cầu người dùng nhập vào một số tự nhiên trong khoảng từ 1 đến 10, sử dụng câu lệnh switch để in ra dạng số La Mã tương ứng của số đó. */

```
#include <iostream>
using namespace std;
int main(){
    int so;
    cout << "Nhập so (1-10): ";
    cin >> so;
    switch (so){
        case 1: cout << "I"; break;
        case 2: cout << "II"; break;
        case 3: cout << "III"; break;
        case 4: cout << "IV"; break;
        case 5: cout << "V"; break;
        case 6: cout << "VI";
        case 7: cout << "VII"; break;
        case 8: cout << "VIII"; break;
        case 9: cout << "IX"; break;
        case 10: cout << "X"; break;
        default: cout << "Khong hop le";
    }
    cout << endl;
    return 0;
}
```

Nhap so (1-10): 6
VIVII

Nếu không có break, sẽ chạy sang cả lệnh của nhãn ở dưới, cho đến khi nào gặp break; mới dừng và kết thúc switch

Cách chạy lệnh switch



Cách chạy lệnh switch

1. Choice được đọc giá trị (nếu là biến) hoặc tính giá trị (nếu là biểu thức)
2. Giá trị này được so khớp lần lượt từ *value1* đến *valuen*.
3. Nếu khớp bất kỳ *valuei* nào, chương trình rẽ vào nhãn đó và thực thi từ đây cho đến hết khối switch hoặc đến khi gặp lệnh break
4. Nếu không có giá trị khớp, chương trình rẽ nhánh vào nhãn *default*:

Thực hành: Tận dụng default

```
/* Một hiệu sách tính điểm thưởng cho người mua tùy theo số sách khách hàng đã mua như sau:
    0 cuốn sách đạt 0 điểm
    1 cuốn sách đạt 5 điểm
    2 cuốn sách đạt 15 điểm
    3 cuốn sách đạt 30 điểm
    từ 4 cuốn sách trở lên đạt 60 điểm
Viết chương trình nhập vào số sách của 1 khách hàng và in ra số điểm thưởng của họ là bao nhiêu. */
#include <iostream>
using namespace std;
int main()
{
    int book, diem;
    cout << "Số sách: ";
    cin >> book;
    switch (book)
    {
        case 0: diem = 0; break;
        case 1: diem = 5; break;
        case 2: diem = 15; break;
        case 3: diem = 30; break;
        default: if (book >=4) diem = 60;
    }
    cout << diem << " điểm\n";
    return 0;
}
```

Các nhãn thường chỉ khớp giá trị đơn lẻ, riêng default có thể khớp với một miền nhiều giá trị

Lưu ý khi dùng switch

- Choice có thể là biến, hằng, biểu thức, còn nhãn phải là giá trị hằng, tức có thể là literal, hằng đặt tên, hoặc biểu thức hằng
- Choice và nhãn phải cùng kiểu, là các kiểu chuyển chính xác được về dạng int
- Một nhãn có thể đi cùng nhiều lệnh mà không cần {}, ngoại lệ là khi có lệnh khai báo biến thì phải có {}
 - Lý do nên để mọi lệnh khai báo biến mới ở ngoài switch
- Thực hành: Các đoạn mã sau có lỗi không, vì sao?

```
int choice;  
cin >> choice;  
switch(choice){  
|    // ...  
}
```

OK

```
char choice;  
cin >> choice;  
switch(choice){  
|    // ...  
}
```

OK

```
string choice;  
cin >> choice;  
switch(choice){  
|    // ...  
}
```

error

```
float choice;  
cin >> choice;  
switch(choice){  
|    // ...  
}
```

error

```
bool choice;  
switch(choice){  
|    // ...  
}
```

OK

Lưu ý khi dùng switch

```
switch(2){  
    case 1: cout << "Do something"; break;  
    case 2: cout << "Do another thing"; break;  
}
```

Được dùng cả hằng, biến, lẫn biểu thức kiểu int làm choice

```
int num1, num2;  
cin >> num1 >> num2;  
switch(num1+num2){  
    case 1: cout << "1 \n"; break;  
    case 2: cout << "2 \n"; break;  
}
```

Nhap lua chon 0 1
1

```
char choice;  
cin >> choice;  
switch(choice){  
    case "A": cout << "Do something"; break;  
    case "B": cout << "Do another thing"; break;  
}
```

Lỗi: Nhãn không thể là xâu, và phải cùng kiểu với choice

```
bool choice;  
switch(choice){  
    case 1: cout << "Do something"; break;  
    case 2: cout << "Do another thing"; break;  
}
```

Chạy được song không nhiều ý nghĩa, vì với biến bool dùng if/else tiện hơn. Ở đây cũng thiếu giá trị 0

```
cin >> choice >> num1 >> num2;  
switch(choice){  
    case num1: cout << "Do something"; break;  
    case num2: cout << "Do another thing"; break;  
}
```

Lỗi: nhãn cần là hằng hay biểu thức hằng

Ứng dụng của các câu lệnh điều kiện

- ❑ Xác minh đầu vào
- ❑ Tạo thực đơn
- ❑ Biến cò

Xác minh đầu vào

- Kiểm tra dữ liệu đầu vào hợp lệ hay không
- Đầu vào sai có thể làm chương trình thành vô nghĩa
- VD: ngoài đúng kiểu dữ liệu, còn có thể kiểm tra:
 - ❑ Phạm vi: tháng không thể >12
 - ❑ Tính hợp lý: số giờ làm việc trong tuần cần <60
 - ❑ Thuộc tập cho trước: là các lựa chọn menu
 - ❑ Giá trị không được gây lỗi: mẫu số phải khác 0

Ví dụ: Xác minh đầu vào

```
// Nhập vào một số x đại diện cho điểm số, in ra dạng điểm chữ tương ứng :
// Chú ý: kiểm tra đầu vào, không chấp nhận các số nhỏ hơn 0 hoặc lớn hơn 100 */
#include <iostream>
using namespace std;
int main()
{
    const int A_SCORE(90),
             B_SCORE(80),
             C_SCORE(70),
             D_SCORE(60),
             MIN_SCORE(0),    // giá trị nhỏ nhất hợp lệ
             MAX_SCORE(100);  // giá trị lớn nhất hợp lệ

    int score;
    cout << "Diem so: ";
    cin  >> score;

    if (score >= MIN_SCORE && score <= MAX_SCORE){ // kiểm tra hợp lệ
        if (score >= A_SCORE)
            cout << "A";
        else if (score >= B_SCORE)
            cout << "B";
        else if (score >= C_SCORE)
            cout << "C";
        else if (score >= D_SCORE)
            cout << "D";
        else
            cout << "F";
        cout << endl;
    }
    else
        cout << "Diem can trong khoang [" << MIN_SCORE << ", " << MAX_SCORE << "]\n";
    return 0;
}
```

Thực hành: Xác minh đầu vào

```
// Nhập vào cân nặng, tính trọng lượng = cân nặng * 9.8
// Nếu trọng lượng < 10, thông báo vật quá nhẹ
// Nếu trọng lượng > 100, thông báo vật quá nặng
// Chú ý: kiểm tra đầu vào không chấp nhận các số nhỏ hơn 0
```

```
#include<iostream>
using namespace std;
int main(){
    const float G(9.8);
    float khoiluong, trongluong;
    cout << "Khoi luong: ";
    cin >> khoiluong;

    if (khoiluong<0) cout << "Khong hop le\n";
    else{
        trongluong = khoiluong * G;
        if (trongluong<10) cout << "Vat qua nhẹ \n";
        if (trongluong>100) cout << "Vat qua nặng \n";
    }
    return 0;
}
```

Nếu được hãy xử lý trường hợp lỗi trước, vì thường chỉ vài dòng lệnh

Thực hành: Xác minh đầu vào

/* Yêu cầu người dùng nhập vào một số tự nhiên trong khoảng từ 1 đến 10, sử dụng câu lệnh switch để in ra dạng số La Mã tương ứng của số đó.
Chú ý: kiểm tra đầu vào, không chấp nhận các số nhỏ hơn 1 hoặc lớn hơn 10 */

```
#include <iostream>
using namespace std;
int main(){
    int so;
    cout << "Nhập số (1-10): ";
    cin >> so;
    if (so<1 || so>10)
        cout << "Khong hop le\n";
    else{
        switch (so){
            case 1: cout << "I"; break;
            case 2: cout << "II"; break;
            case 3: cout << "III"; break;
            case 4: cout << "IV"; break;
            case 5: cout << "V"; break;
            case 6: cout << "VI"; break;
            case 7: cout << "VII"; break;
            case 8: cout << "VIII";break;
            case 9: cout << "IX"; break;
            case 10: cout << "X"; break;
            default: cout << "Khong hop le";
        }
        cout << endl;
    }
    return 0;
}
```

Thực đơn

- Thực đơn là một danh sách các chức năng mà người lập trình hiển thị cho người dùng lựa chọn, tùy người dùng chọn chức năng gì mà chương trình sẽ thực hiện theo tương ứng
- Có thể cài đặt dùng if/else if, nhưng hiệu quả hơn với switch
- Ý tưởng: Đánh số/chữ ứng với các chức năng rồi nhập từ bàn phím một biến kiểu số/chữ để thể hiện lựa chọn chức năng của người dùng.

Dùng switch để tạo thực đơn

- Ta dùng switch tạo thực đơn theo các bước sau:
 - Hiển thị thực đơn, gán số/chữ cho từng lựa chọn
 - Người dùng nhập lựa chọn dạng số/chữ
 - Đọc (cin) lựa chọn này vào một biến kiểu số/chữ
 - Cho biến này là mục choice trong câu lệnh switch
 - Cho các số/chữ ứng với các lựa chọn thực đơn là các nhãn *value_i* ở các cases
 - Mỗi case gồm các lệnh thực thi chức năng tương ứng

Ví dụ: Dùng switch tạo thực đơn

```
#include <iostream>
using namespace std;
int main(){
    float r;
    cout << "Ban kinh:";
    cin >> r;

    int c;
    cout << "Ban muon lam gi: \n"
    << "1. Tinh duong kinh hinh tron\n"
    << "2. Tinh chu vi hinh tron\n"
    << "3. Tinh dien tich hinh tron\n"
    << "4. Thoat\n";
    cin >> c;

    switch(c){          // so khớp lựa chọn
        case 1: cout << "Duong kinh: " << 2 * r << endl; break;
        case 2: cout << "Chu vi: " << 2 * 3.14 * r << endl; break;
        case 3: cout << "Dien tich: " << 3.14 * r * r << endl; break;
        case 4: cout << "Thoat chuong trinh ... " << endl;
                exit(0);
        default: cout << "Lua chon khong ton tai" << endl;
    }
    return 0;
}
```

Thực hành: Dùng switch tạo thực đơn

```
int main(){
    float a, b;
    cout << "Nhap hai so: ";
    cin >> a >> b;

    int c;
    cout << "Ban muon lam gi:\n"
    << "1. Tinh tong \n"
    << "2. Tinh hieu \n"
    << "3. Tinh tich \n"
    << "4. Tinh thuong \n"
    << "5. Thoat \n";
    cin >> c;

    switch (c){        // so khop lua chon
        case 1: cout << "Tong hai so: " << a + b << endl; break;
        case 2: cout << "Hieu hai so: " << a - b << endl; break;
        case 3: cout << "Tich hai so: " << a * b << endl; break;
        case 4: if (!b) cout << "Mau so bang 0, khong the chia\n";
                else cout << "Thuong hai so: " << a + b << endl;
                break;
        case 5: cout << "Thoat chuong trinh ... " << endl;
                exit(0);
        default: cout << "Lua chon khong ton tai " << endl;
    }
    return 0;
}
```


Thực hành: Nếu trong switch có khai báo biến?

```
#include <iostream>
#include <cmath>
using namespace std;

int main(){
    int c;
    cout << "Ban muon lam gi: \n"
    << "1. Tinh e^x \n"
    << "2. Tinh ln x \n"
    << "3. Tinh x^n \n"
    << "4. Thoat \n";
    cin >> c;
```

```
switch (c){
    case 1: {
        float x;
        cout << "x: ";
        cin >> x;
        cout << "e^x: " << exp(x) << endl;
        break;
    }
    case 2: {
        float x;
        cout << "x: ";
        cin >> x;
        cout << "lnx: " << log(x) << endl;
        break;
    }
    case 3: {
        float x;
        int n;
        cout << "x va n: ";
        cin >> x >> n;
        cout << "x^n: " << pow(x, n) << endl;
        break;
    }
    case 4: cout << "Thoat chuong trinh ... \n";
            exit(0);
    default: cout << "Lua chon khong ton tai \n";
}
```



```

switch (c){
    case 1: {
        float x;
        cout << "x: ";
        cin >> x;
        cout << "e^x: " << exp(x) << endl;
        break;
    }
    case 2: {
        float x;
        cout << "x: ";
        cin >> x;
        cout << "lnx: " << log(x) << endl;
        break;
    }
    case 3: {
        float x;
        int n;
        cout << "x va n: ";
        cin >> x >> n;
        cout << "x^n: " << pow(x, n) << endl;
        break;
    }
    case 4: cout << "Thoat chuong trinh ... \n";
            exit(0);
    default: cout << "Lua chon khong ton tai \n";
}

```



```

float x;
int n;
switch (c){
    case 1: cout << "x: ";
            cin >> x;
            cout << "e^x: " << exp(x) << endl;
            break;
    case 2: cout << "x: ";
            cin >> x;
            cout << "lnx: " << log(x) << endl;
            break;
    case 3: cout << "x va n: ";
            cin >> x >> n;
            cout << "x^n: " << pow(x, n) << endl;
            break;
    case 4: cout << "Thoat chuong trinh ... \n";
            exit(0);
    default: cout << "Lua chon khong ton tai \n";
}

```

Khai báo biến ngoài switch làm mã gọn hơn nhiều, nó cũng giúp tránh lỗi khi quên {}

Cờ

- Có lúc ta muốn khảo sát trạng thái chương trình
 - Hoàn cảnh X đã từng xảy ra chưa? Bao lần? Khi nào? Cách nào?
- Giải pháp: Sinh ra một biến mô tả trạng thái chương trình:
 - mỗi khi hoàn cảnh đặc biệt X nào đó xảy ra, ta cập nhật biến bằng một giá trị x tương ứng
 - về sau nếu đọc biến trên thấy bằng x , ta suy ra X từng xảy ra
- Một biến hoạt động như trên gọi là biến cờ (flag), thường là kiểu bool
 - Ban đầu cờ hiệu đang hạ, mỗi khi có một sự kiện đặc biệt, ta dựng cờ hiệu lên, về sau chỉ cần xem cờ hiệu đang hạ hay đang dựng là biết sự kiện từng xảy ra hay chưa

Ví dụ: Cờ

// Nhập từ bàn phím một chuỗi số nguyên, kết thúc bởi -1, in ra xem liệu có
// lúc nào người dùng nhập vào 2 số âm liên tiếp khác -1 trong dãy số chưa.

```
#include <iostream>
#include <iostream>
using namespace std;
int main(){
    bool found1 = false, found2 = false;
    int n;
    do{
        cout << "n: ";
        cin >> n;
        if (n != -1 && !found2){
            if (n < 0){
                if (found1) found2 = true;
                else found1 = true;
            }
            else found1 = false;
        }
    }while (n!=-1);
    if (found2) cout << "Tung nhap vao 2 so am lien tiep\n";
    else cout << "Chua tung nhap 2 so am lien tiep\n";
    return 0;
}
```

Nếu vừa đọc thêm số âm, mà found1 đã dựng sẵn từ trước, thì dựng tiếp found2

cờ found1 dựng ngay khi vừa có một số âm và hạ ngay khi vừa có số không âm.

Chủ đề mở rộng

- ❑ So sánh ký tự
- ❑ So sánh sâu

So sánh các ký tự

- Chính là so sánh mã ASCII của chúng
 - 'A' < 'B' vì mã ASCII của 'A' (65) nhỏ hơn mã ASCII của 'B' (66)
 - '1' < '2' vì mã ASCII của '1' (49) nhỏ hơn mã ASCII của '2' (50)
- Không cần nhớ mã ASCII của các ký tự, chỉ thuộc vài quy tắc:
 - Chữ thường luôn có mã ASCII lớn hơn chữ hoa, vd 'a' > 'Z'
 - Trong chữ thường (chữ hoa), mã tăng dần theo chiều abc
 - Trong chữ số, mã tăng dần theo chiều 0 đến 9
 - Nếu muốn xem thử giá trị mã ascii của một ký tự:

```
int test = 'A';
```

```
cout << test; // in ra 65
```

Ví dụ: So sánh ký tự

```
// các ký tự cũng có thể so sánh với nhau qua toán tử quan hệ
#include <iostream>
using namespace std;
int main()
{
    char ch;
    cout << "Nhap chu cai hoac chu so: ";
    ch = cin.get();

    if (ch >= '0' && ch <= '9')
        cout << "Ban nhap vao chu so.\n";
    else if (ch >= 'A' && ch <= 'Z')
        cout << "Ban nhap chu cai viet hoa.\n";
    else if (ch >= 'a' && ch <= 'z')
        cout << "Ban nhap chu cai viet thuong.\n";
    else
        cout << "Day khong phai chu cai hay chu so.\n";

    return 0;
}
```

So sánh các chuỗi (chỉ cho kiểu string)

- Dựa trên so sánh ký tự: Đi từ đầu hai chuỗi và so sánh mã ASCII các ký tự tương ứng của chúng

```
string name1 = "Mary";  
string name2 = "Mark";
```

Để hai chuỗi bằng nhau thì ít nhất chúng phải có số ký tự bằng nhau

```
name1 > name2 // true
```

```
name1 <= name2 // false
```

```
name1 != name2 // true
```

Nếu chuỗi s1 ngắn hơn chuỗi s2 nhưng bắt đầu bằng một ký tự lớn hơn thì sao??

```
name1 < "Mary Jane" // true
```

```
"Me" > "Mary" // true
```

Ví dụ: So sánh xâu

// các xâu cũng có thể so sánh với nhau qua toán tử quan hệ

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;
int main()
{
    const double PRICE_A = 249.0,
    |         |         | PRICE_B = 299.0;
    string code; // mã sản phẩm
    cout << "Ma san pham S-29A co gia 249.0 \n"
    |     << "Ma san pham S-29A co gia 299.0 \n";
    cout << "Ma san pham ban muon mua:";
    cin  >> code;

    cout << fixed << showpoint << setprecision(2);

    if (code == "S-29A")
    |     cout << "Gia la " << PRICE_A << " $ \n";
    else if (code == "S-29B")
    |     cout << "Gia la " << PRICE_B << " $ \n";
    else
    |     cout << code << " khong phai ma hop le \n";
    return 0;
}
```


Tổng kết về các câu lệnh điều kiện

- Toán tử quan hệ, toán tử logic, các thao tác kiểu bool
- Các kiểu if: if, if/else, biểu thức điều kiện, if/else if
- Lệnh switch
- Một số ứng dụng: xác minh đầu vào, tạo thực đơn, biến cờ
- Mở rộng: toán tử quan hệ dùng so sánh xâu và so sánh ký tự

Trò chơi đoán số (tiếp)

Máy tính nghĩ ra một số ngẫu nhiên trong khoảng từ 1 đến 1000, nó giữ bí mật và mời bạn đoán. Mỗi lần đoán, nó sẽ cho bạn thắng nếu bạn trả lời đúng số cần đoán. Nếu không đúng, bạn được biết giá trị vừa đoán là quá cao hay quá thấp so với số cần đoán. Sau 10 đoán mà vẫn không đoán trúng thì bạn sẽ thua.

Hello! What is your name?

Mai

Play with me, Mai! I am thinking of a number between 1 and 1000.

Take a guess.

10

Your guess is too high.

Take a guess.

2

Your guess is too low.

Take a guess.

4

Well done, Mai! You guessed my number in 3 guesses!

Trò chơi đoán số - Tuần 3

- Tuần này ta có thể mở rộng lời chào tuần trước để đưa ra thông báo về lần đoán của người chơi:
 - Nếu họ đoán số cao hơn số máy sinh ra, hãy in ra thông báo đoán số quá cao, mời họ đoán lại
 - Nếu họ đoán số thấp hơn số máy sinh ra, hãy in ra thông báo đoán số quá thấp, mời họ đoán lại
 - Nếu họ đoán đúng, hãy in ra chúc mừng họ đã đoán đúng số máy tính nghĩ.
 - Sinh ra một biến cò lưu lại người dùng đã đoán đúng chưa
- Hãy cập nhật chương trình GuessNumber đã làm tuần 2 để bổ sung mở rộng trên